Stripe and PayPal to BigQuery

Monthly Automation Pipeline Plan

Junior developer view - flow, access, scheduling, storage, and BigQuery design

Prepared: 2026-06-12

```
Main goal:
Automatically extract monthly Stripe and PayPal data using limited source-platform access,
store raw files in Cloud Storage, load raw data to BigQuery, transform to reporting tables,
and keep run logs and alerts for every monthly execution.
```

Area	Decision
Compute	Cloud Run Job running a Python backend extractor
Schedule	Cloud Scheduler runs the job monthly, normally on the 3rd or 5th day
Secrets	Google Secret Manager stores Stripe key and PayPal client credentials
Raw storage	Cloud Storage stores original API response files by source/year/month/run_id
Warehouse	BigQuery raw, staging, final fact, and ETL log tables
Stripe source access	Restricted API key with read permissions only
PayPal source access	REST app with Transaction Search permission, using temporary OAuth tokens

Security reminder: This document uses placeholder commands. Never paste real Stripe keys, PayPal client secrets, or access tokens into GitHub, chat, logs, or screenshots.

1. End-to-end pipeline overview

The pipeline should run without manual monthly downloads. The backend job calculates the month to extract, calls Stripe and PayPal APIs, writes raw files to Cloud Storage, loads raw rows to BigQuery, then runs SQL transformations to create final finance tables.

```
Cloud Scheduler
      |
      v
Cloud Run Job - Python backend
      |
      +--> Secret Manager - read API credentials
      |
      +--> Stripe API - read charges and refunds
      |
      +--> PayPal API - create OAuth token and read transactions
      |
      v
Cloud Storage - raw JSON/NDJSON backup files
      |
      v
BigQuery raw tables
      |
      v
BigQuery staging tables - clean and standardize
      |
      v
BigQuery final fact table - combined reporting data
      |
      v
Run log + alert - email/Slack on success or failure
```

Step	What happens	Owner component
1	Monthly trigger starts the extraction job.	Cloud Scheduler
2	Backend calculates previous month window and run_id.	Cloud Run Job
3	Backend reads Stripe restricted key and PayPal credentials.	Secret Manager
4	Backend extracts Stripe charges and refunds with pagination.	Stripe API
5	Backend creates PayPal OAuth token and extracts transaction pages.	PayPal API
6	Raw responses are saved as ndJSON files.	Cloud Storage
7	Raw files are loaded to raw BigQuery tables.	BigQuery load jobs
8	SQL transforms raw rows into staging and final tables.	BigQuery SQL
9	The run writes row counts, status, and errors to a log table.	BigQuery etl_run_log
10	The job sends success or failure notification.	Cloud Run + alert integration

2. Access model and credentials

2.1 Stripe access

For Stripe, the pipeline uses a restricted API key. In test mode it starts with rk_test_. In live mode it starts with rk_live_. The key should only be used by the backend job, not by frontend code.

Stripe permission	Set to	Why
Balance	Read	Allows balance-level data checks and balance-related objects.
Balance Transaction Sources	Read	Allows expanded balance_transaction details from charges and refunds.

Charges and Refunds	Read	Allows GET /v1/charges and GET /v1/refunds.
All write permissions	None	The pipeline must not create payments, refunds, payouts, or customer changes.
Customer/payment method data	None initially	Add only if reporting later needs customer fields.

Stripe design: For your stated Stripe requirement, the backend should pull charges and refunds, not generate Stripe reports. This keeps source access read-only.

2.2 PayPal access

For PayPal, the pipeline stores Client ID and Secret from a PayPal Sandbox or Live REST app. Each run requests a temporary OAuth access token and uses it to call Transaction Search. The access token is not stored permanently.

PayPal setting	Required value	Why
App type	Merchant	The business account owns the transactions.
Environment	Sandbox for testing, Live for production	Sandbox and live credentials are separate.
Permission	Transaction Search enabled	Required for GET /v1/reporting/transactions.
Expected token scope	reporting/search/read	Confirms the token can read reporting transactions.
Stored secrets	PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET	Cloud Run exchanges these for temporary tokens.

PayPal propagation: If Transaction Search is enabled after the first token was generated, PayPal may take up to 9 hours before the new scope appears on a new token. Keep generating a new token until the scope includes reporting/search/read.

2.3 GCP service account access

Resource	Minimum role/permission idea
Secret Manager	Access secrets for Stripe and PayPal credentials
Cloud Storage bucket	Write raw files and read them for BigQuery load jobs
BigQuery dataset	Create jobs, update raw tables, run transformations
Cloud Run Job	Execute the containerized extractor
Cloud Scheduler	Invoke the Cloud Run Job

3. Monthly scheduling flow

The monthly job should run after the month is closed. A safe default is the 3rd day or 5th day of every month. Use an off-hour schedule such as 06:17 UTC to avoid exact-hour duplication concerns in some scheduled systems.

```
Example schedule:
Run: 3rd day of each month, 06:17 UTC
Job input: none by default
Backend calculates: previous full calendar month

Example:
Run date: 2026-06-03
Extract month: 2026-05
Window start: 2026-05-01 00:00:00 UTC
Window end:   2026-06-01 00:00:00 UTC
```

Mode	How it works
Normal monthly run	No date input. Job extracts the previous complete month.
Manual rerun	Pass RUN_MONTH=YYYY-MM to re-extract one month.
Backfill	Run the job multiple times with RUN_MONTH for old months.
Idempotent rerun	Use source + transaction_id + month keys and MERGE logic so reruns do not duplicate final data.

4. Stripe extraction flow

Stripe extraction is split into two API pulls: charges and refunds. This exactly supports your requirement for charge denominator and refund numerator calculations.

4.1 Stripe charges API call

```
GET https://api.stripe.com/v1/charges
Auth: Basic auth username = rk_test_or_rk_live_key, password blank
Params:
  created[gte] = month_start_epoch
  created[lt]  = month_end_epoch
  limit = 100
  expand[] = data.balance_transaction
```

Stripe test command

```
Example for May 2026:
curl -G https://api.stripe.com/v1/charges   -u "rk_test_OR_rk_live_KEY:"   -d "created[gte]=1777593600"
-d "created[lt]=1780272000"   -d "limit=100"   -d "expand[]=data.balance_transaction"
```

Stripe charge field	Purpose
id	Dedup key and join key to refunds.
created	Month attribution. Convert Unix seconds to TIMESTAMP.
status	Keep succeeded charges for final finance logic.
captured	Keep captured = true for final denominator.
currency	Currency code such as aud, usd, gbp.
balance_transaction.id	Trace back to Stripe balance movement.
balance_transaction.amount	Gross charge amount in smallest currency unit.
balance_transaction.fee	Stripe fee in smallest currency unit.
balance_transaction.net	Net amount after fee.

4.2 Stripe refunds API call

```
GET https://api.stripe.com/v1/refunds
Auth: Basic auth username = rk_test_or_rk_live_key, password blank
Params:
  created[gte] = month_start_epoch
  created[lt]  = month_end_epoch
  limit = 100
  expand[] = data.balance_transaction
```

Stripe refund test command

```
Example for May 2026:
curl -G https://api.stripe.com/v1/refunds   -u "rk_test_OR_rk_live_KEY:"   -d "created[gte]=1777593600"
-d "created[lt]=1780272000"   -d "limit=100"   -d "expand[]=data.balance_transaction"
```

Stripe refund field	Purpose
id	Refund dedup key.
charge	Parent charge ID. Join to charges.id for attribution.
created	Refund month attribution.
status	Keep succeeded refunds for final refund logic.
currency	Currency code.
balance_transaction.id	Refund balance movement ID.
balance_transaction.amount	Refund amount, usually negative, in smallest currency unit.
balance_transaction.fee	Fee adjustment if returned by Stripe. Often zero or negative depending on account rules.
balance_transaction.net	Net impact on Stripe balance.

4.3 Stripe pagination

Stripe list endpoints return up to 100 records per call. If has_more is true, pass starting_after as the last object ID from the previous page.

```
Pseudo-code:
starting_after = None
```

```
while True:
    response = GET /v1/charges with limit=100 and starting_after
    write each object to raw file
    if response.has_more is false: break
    starting_after = response.data[-1].id
```

4.4 Stripe storage output

```
GCS paths:
gs://payment-raw/stripe/charges/year=2026/month=05/run_id=20260603T061700Z/data.ndjson
gs://payment-raw/stripe/refunds/year=2026/month=05/run_id=20260603T061700Z/data.ndjson

Each line is one JSON object plus metadata fields:
_source = "stripe"
_endpoint = "charges" or "refunds"
_run_id = "..."
_extract_month = "2026-05"
_loaded_at = current timestamp
```

5. PayPal extraction flow

PayPal extraction uses two steps: first create an OAuth access token from Client ID and Secret, then call Transaction Search. The backend should not save the temporary token after the run.

5.1 Generate PayPal OAuth token

```
POST https://api-m.sandbox.paypal.com/v1/oauth2/token   # sandbox
POST https://api-m.paypal.com/v1/oauth2/token           # live

Auth:
  Basic auth username = PAYPAL_CLIENT_ID
  Basic auth password = PAYPAL_CLIENT_SECRET

Body:
  grant_type=client_credentials
```

PayPal token test command

```
curl -X POST "https://api-m.sandbox.paypal.com/v1/oauth2/token"   -u
"PAYPAL_CLIENT_ID:PAYPAL_CLIENT_SECRET"   -H "Content-Type: application/x-www-form-urlencoded"   -d
"grant_type=client_credentials"
```

5.2 Validate token scope

Before calling Transaction Search, check that the token response scope includes reporting/search/read. If the scope is missing, the API will return NOT_AUTHORIZED or insufficient permissions.

```
Expected token response shape:
{
  "scope": "... https://uri.paypal.com/services/reporting/search/read ...",
  "access_token": "A21AA...",
  "token_type": "Bearer",
  "expires_in": 32400
}
```

5.3 PayPal Transaction Search API call

```
GET https://api-m.sandbox.paypal.com/v1/reporting/transactions   # sandbox
GET https://api-m.paypal.com/v1/reporting/transactions           # live

Headers:
```

```
  Authorization: Bearer ACCESS_TOKEN

Params:
  start_date = 2026-05-01T00:00:00Z
  end_date = 2026-05-31T23:59:59Z
  fields = all
  page_size = 500
  page = 1, 2, 3...
  balance_affecting_records_only = Y
```

PayPal transaction test command

```
curl -X GET "https://api-m.sandbox.paypal.com/v1/reporting/transactions?start_date=2026-05-01T00:00:00Z&end_date=2026-05-31T23:59:59Z&fields=all&page_size=500&page=1&balance_affecting_records_only=Y" -H "Authorization: Bearer ACCESS_TOKEN"
```

PayPal transaction field	Purpose
transaction_info.transaction_id	Base transaction identifier. Do not use alone as the only dedup key.
transaction_info.transaction_event_code	Event type, used to classify payment/refund/fee/etc.
transaction_info.transaction_initiation_date	Original transaction time.
transaction_info.transaction_updated_date	Last updated time. Useful for dedup key and audit.
transaction_info.transaction_amount.currency_code	Currency.
transaction_info.transaction_amount.value	Gross amount as decimal string.
transaction_info.fee_amount.value	PayPal fee as decimal string if available.
transaction_info.transaction_status	Status such as completed, pending, denied, reversed.
payer_info.email_address	Optional payer email, only if business reporting needs it.
cart_info.item_details	Optional item-level detail when present.

5.4 PayPal pagination and limits

Use page_size=500 and loop from page=1 to total_pages. PayPal Transaction Search supports a maximum 31-day date range per API call, so one calendar month is normally OK; if the month or backfill window is too large, split into smaller chunks.

```
Pseudo-code:
page = 1
while True:
    response = GET /v1/reporting/transactions with page_size=500 and page
    write each transaction_details item to raw file
    if page >= response.total_pages: break
    page += 1

If the API returns too many records for a date range:
    split the month into smaller chunks, for example 7-day windows.
```

5.5 PayPal storage output

```
GCS path:
gs://payment-raw/paypal/transactions/year=2026/month=05/run_id=20260603T061700Z/data.ndjson

Each line is one transaction_details item plus metadata fields:
_source = "paypal"
_endpoint = "transactions"
_run_id = "..."
_extract_month = "2026-05"
_loaded_at = current timestamp
```

6. Cloud Storage and BigQuery design

6.1 Cloud Storage structure

```
Bucket: gs://payment-raw

stripe/
  charges/year=YYYY/month=MM/run_id=RUN_ID/data.ndjson
  refunds/year=YYYY/month=MM/run_id=RUN_ID/data.ndjson
paypal/
  transactions/year=YYYY/month=MM/run_id=RUN_ID/data.ndjson

Why keep raw files:
  - Audit trail
  - Reprocessing
  - Debugging
  - Recovery if BigQuery load fails
```

6.2 BigQuery dataset and tables

Layer	Table name	Purpose
Raw	finance_payments.raw_stripe_charges	Stores original Stripe charge JSON rows plus run metadata.
Raw	finance_payments.raw_stripe_refunds	Stores original Stripe refund JSON rows plus run metadata.
Raw	finance_payments.raw_paypal_transactions	Stores original PayPal transaction_details JSON rows plus run metadata.
Staging	finance_payments.stg_stripe_charges	Flattened and typed Stripe charges.
Staging	finance_payments.stg_stripe_refunds	Flattened and typed Stripe refunds.
Staging	finance_payments.stg_paypal_transactions	Flattened and typed PayPal transactions.
Final	finance_payments.fact_payment_transactions	Combined Stripe + PayPal reporting table.
Log	finance_payments.etl_run_log	One row per source per run with status, counts, and errors.

6.3 Raw table pattern

Column	Type	Meaning
run_id	STRING	Unique job run ID.
extract_month	STRING	YYYY-MM month being extracted.
source	STRING	stripe or paypal.
endpoint	STRING	charges, refunds, or transactions.
raw_json	JSON or STRING	Original API object. Use JSON type if available in your standard.
ingested_at	TIMESTAMP	When row was loaded to BigQuery.

6.4 Staging table patterns

Table	Important columns
stg_stripe_charges	charge_id, created_ts, charge_month, status, captured, currency, gross_minor, fee_minor, net_minor, gross_amount, fee_amount, net_amount, balance_transaction_id
stg_stripe_refunds	refund_id, charge_id, created_ts, refund_month, status, currency, refund_minor, fee_minor, net_minor, refund_amount, balance_transaction_id
stg_paypal_transactions	paypal_unique_key, transaction_id, event_code, initiation_ts, updated_ts, status, currency, gross_amount, fee_amount, net_amount

6.5 Final fact table pattern

Column	Meaning
payment_source	stripe or paypal.
transaction_id	Charge ID, refund ID, or PayPal transaction ID depending on row type.
parent_transaction_id	For Stripe refunds, the parent charge ID; for PayPal, reference ID

	if available.
transaction_type	charge, refund, fee, payout, adjustment, etc.
transaction_date	Date of transaction.
transaction_month	YYYY-MM based on transaction_date.
status	Normalized status.
currency	Currency code.
gross_amount	Decimal amount in major units.
fee_amount	Decimal fee amount in major units.
net_amount	Decimal net amount in major units.
source_unique_key	Stable unique key used for MERGE deduplication.
run_id	Last run that loaded/updated the record.
loaded_at	Load timestamp.

7. BigQuery load and transformation flow

The recommended first version lets Cloud Run write files to Cloud Storage, then immediately starts BigQuery load jobs for those files. Transformations can run either inside Cloud Run after the load, or as BigQuery scheduled queries. For simplicity and control, start with Cloud Run triggering the SQL after successful raw loads.

```
Cloud Run job logic:
1. Extract Stripe charges -> write GCS file -> load raw_stripe_charges.
2. Extract Stripe refunds -> write GCS file -> load raw_stripe_refunds.
3. Extract PayPal transactions -> write GCS file -> load raw_paypal_transactions.
4. Run SQL to build/merge staging tables.
5. Run SQL to merge into fact_payment_transactions.
6. Write etl_run_log status and row counts.
```

7.1 Idempotent MERGE strategy

The pipeline must be safe to rerun. Do not just INSERT into final tables without a dedup key. Use MERGE based on source_unique_key.

```
Stripe charge key:
  source_unique_key = "stripe_charge:" || charge_id

Stripe refund key:
  source_unique_key = "stripe_refund:" || refund_id

PayPal transaction key:
  source_unique_key = MD5(transaction_id || event_code || updated_date || amount || currency)

Final load pattern:
  MERGE fact_payment_transactions T
  USING staging_new_rows S
  ON T.source_unique_key = S.source_unique_key
  WHEN MATCHED THEN UPDATE SET ...
  WHEN NOT MATCHED THEN INSERT ...
```

7.2 Stripe refund leakage calculation

Your Stripe leakage/refund logic uses charges as the denominator and refunds as the numerator. The join is refunds.charge = charges.id. Keep the raw balance_transaction fee on the charge if you want fee/leakage analysis.

Metric	Formula idea
Charge denominator	Sum Stripe charge balance_transaction.amount for succeeded and captured charges.
Refund numerator	Absolute value of sum Stripe refund balance_transaction.amount for succeeded refunds.
Refund fee/leakage optional	Use charge balance_transaction.fee if comparing original fee paid against refund amount.
Month attribution option A	Use charge.created month for charge denominator.
Month attribution option B	Use refund.created month for refund numerator.
Join logic	Stripe refund.charge = Stripe charge.id.

8. Backend implementation plan

Use a small Python backend. Keep extractors separate so Stripe and PayPal failures can be logged clearly.

```
Suggested repository structure:
/payment_pipeline
  main.py
  config.py
  secrets.py
  extractors/
    stripe_extractor.py
    paypal_extractor.py
  storage/
    gcs_writer.py
    bigquery_loader.py
  sql/
    stg_stripe_charges.sql
    stg_stripe_refunds.sql
    stg_paypal_transactions.sql
    merge_fact_payment_transactions.sql
  logging/
    run_logger.py
  Dockerfile
  requirements.txt
```

Module	Responsibility
main.py	Calculate run month, call each extractor, orchestrate load and transform.
secrets.py	Read secrets from Secret Manager; never print secret values.
stripe_extractor.py	Call charges/refunds APIs, paginate, return objects.
paypal_extractor.py	Generate token, validate scope, call transaction search, paginate.
gcs_writer.py	Write ndJSON files to the correct GCS path.
bigquery_loader.py	Start load jobs and execute SQL transformations.
run_logger.py	Write run status, row counts, and error details.

8.1 Main backend pseudo-code

```
def main():
    run_id = now_utc_as_id()
    month = get_run_month_from_env_or_previous_month()
    window = build_month_window(month)

    log_run_start(run_id, month)

    stripe_key = secret("STRIPE_RESTRICTED_API_KEY")
    paypal_client_id = secret("PAYPAL_CLIENT_ID")
    paypal_secret = secret("PAYPAL_CLIENT_SECRET")

    stripe_charges = extract_stripe_charges(stripe_key, window)
    stripe_refunds = extract_stripe_refunds(stripe_key, window)
    paypal_txns = extract_paypal_transactions(paypal_client_id, paypal_secret, window)

    gcs_paths = write_raw_files(run_id, month, stripe_charges, stripe_refunds, paypal_txns)
    load_raw_files_to_bigquery(gcs_paths)
    run_bigquery_transformations(run_id, month)

    validate_counts(run_id, month)
    log_run_success(run_id, counts)
    send_success_alert(run_id, counts)
```

9. Monitoring, errors, and alerts

Check	Action if failed

Secret read failed	Stop job, log FAILED, alert immediately.
Stripe auth/permission error	Stop Stripe branch, log error code and request_id, alert.
PayPal token scope missing	Stop PayPal branch, log NOT_AUTHORIZED risk, alert.
API pagination failed	Retry with exponential backoff; if still failing, log failed page.
GCS write failed	Stop before BigQuery load; alert.
BigQuery load failed	Keep raw file; log failed load job ID; alert.
Row count zero	Warn, not always failure. Zero is valid if month has no transactions.
Duplicate rows detected	Fail transform or quarantine duplicates before final MERGE.

9.1 ETL run log table

Column	Type
run_id	STRING
source	STRING
extract_month	STRING
window_start	TIMESTAMP
window_end	TIMESTAMP
status	STRING: STARTED, SUCCESS, FAILED, PARTIAL_SUCCESS
rows_extracted	INT64
rows_loaded	INT64
gcs_path	STRING
error_message	STRING
started_at	TIMESTAMP
finished_at	TIMESTAMP

9.2 Alert message example

```
Payment pipeline completed
Run ID: 20260603T061700Z
Month: 2026-05
Stripe charges: 12,450 extracted / 12,450 loaded
Stripe refunds: 310 extracted / 310 loaded
PayPal transactions: 3,210 extracted / 3,210 loaded
Status: SUCCESS
BigQuery final rows merged: 15,970
```

10. Testing plan

Phase	Test
Stripe sandbox key	Use rk_test restricted key to call GET /v1/balance and GET /v1/charges.
Stripe dummy charge	Create dummy test payment using sk_test, then fetch with rk_test.
Stripe dummy refund	Refund the dummy charge using sk_test, then fetch refund with rk_test.
PayPal sandbox token	Generate token with Client ID and Secret; verify reporting/search/read scope.
PayPal transaction API	Call GET /v1/reporting/transactions with sandbox token.
Cloud Run local test	Run Python job locally with environment variables.
GCS test	Confirm raw files are written with correct year/month/run_id path.
BigQuery raw load	Load one small file into raw tables.
Transformation test	Run staging and final MERGE for one month.
Rerun test	Rerun same month and confirm final rows do not duplicate.

11. Production rollout checklist

- Create live Stripe restricted key with the same read-only permissions.
- Create or use live PayPal REST app with Transaction Search enabled.
- Rotate any credentials that were exposed during testing.
- Store all credentials in Secret Manager.
- Create GCS bucket with lifecycle policy if raw files should expire after a retention period.

- Create BigQuery dataset and raw/staging/final/log tables.
- Deploy Cloud Run Job with service account permissions.
- Run one manual production dry run for a small date range if possible.
- Run one full prior-month extraction and compare against dashboard/report totals.
- Enable Cloud Scheduler monthly trigger.
- Enable failure alerts.
- Document runbook for reruns and credential rotation.

12. Open decisions for discussion

Question	Recommended default
Should the final table include customer emails?	No initially. Add later only if reporting needs it.
Should Stripe use charges/refunds or balance_transactions?	Use charges/refunds because your required fields are charge/refund based.
Should PayPal include all records or only balance-affecting records?	Use balance_affecting_records_only=Y for finance reporting.
Which day should the monthly job run?	3rd or 5th day of each month.
Should transformations run inside Cloud Run or BigQuery scheduled queries?	Start inside Cloud Run after raw load for one controlled pipeline.
How long should raw files be retained?	At least 12 months, or longer if finance/audit needs it.

13. Official documentation references

These are the official documentation pages used to design this pipeline. Use them again when implementing because permissions, fields, and UI labels can change over time.

- **Stripe restricted API keys:** https://docs.stripe.com/keys/restricted-api-keys
- **Stripe list charges API:** https://docs.stripe.com/api/charges/list
- **Stripe charge object fields:** https://docs.stripe.com/api/charges/object
- **Stripe list refunds API:** https://docs.stripe.com/api/refunds/list
- **Stripe refund object fields:** https://docs.stripe.com/api/refunds/object
- **PayPal REST API authentication:** https://developer.paypal.com/api/rest/authentication/
- **PayPal Transaction Search integration guide:** https://developer.paypal.com/docs/transaction-search/
- **PayPal Transaction Search API reference:** https://developer.paypal.com/docs/api/transaction-search/v1/
- **Google Cloud Run jobs on a schedule:** https://docs.cloud.google.com/run/docs/execute/jobs-on-schedule
- **Google Secret Manager:** https://docs.cloud.google.com/secret-manager/docs
- **BigQuery load JSON from Cloud Storage:** https://docs.cloud.google.com/bigquery/docs/loading-data-cloud-storage-json
- **BigQuery scheduled queries:** https://docs.cloud.google.com/bigquery/docs/scheduling-queries